



东南大学
SOUTHEAST UNIVERSITY

Chapter 12 Object-Oriented Programming: polymorphism

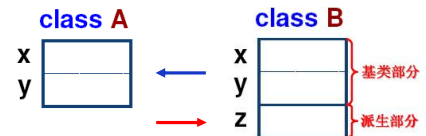


©2025, College of Software Engineering, Southeast University

12.1 Introduction

- class B inherit from class A, i.e. B is a A

1. B big;
2. A small = big;
3. A *pSmall = &big; $\longrightarrow$ B *pBig = (B *) pSmall;
4. A &refSmall = big; $\longrightarrow$ B &refBig = (B &) refSmall;



©2025, College of Software Engineering, Southeast University

12.1 Introduction(cont.)

- Polymorphism(多态)
 - In Biology
 - In OOP
 - Each object in inheritance hierarchies performs the correct tasks for that object's type
 - Different actions occur depending on the type of object
 - Process objects of classes that are part of the same hierarchy as if they are all objects of the base-class

©2025, College of Software Engineering, Southeast University

12.1 Introduction(cont.)

- 用户通过键盘输入多个员工信息，统计薪水数据：
 - (1) CommissionEmployee
name, ssn, grossSales, **commisionRate**
 - (2) BasePlusCommissionEmployee
name, ssn, grossSales, **commisionRate**, **baseSalary**
- 用vector或者array来保存指向员工对象的指针
 - CommissionEmployee Pointer
- 用指针调用相应的earnings()函数计算薪水

©2025, College of Software Engineering, Southeast University

12.1 Introduction(cont.)

- Polymorphism(多态):

通过指向派生类的**基类指针或基类引用**，调用**派生类的函数**；将不同的派生类对象都当作基类来处理，可以**屏蔽不同派生类对象之间的差异**，写出通用代码，实现通用化编程，以适应需求变化
- Virtual Function(虚函数)
- Function Overridden(函数重写)

©2025, College of Software Engineering, Southeast University

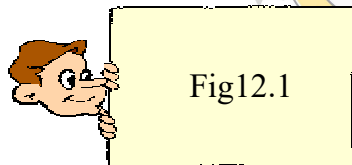
12.3 Relationships Among Objects in an Inheritance Hierarchy Examples

- 12.3.1 Invoking base-class functions from derived-class objects (基类指针指向派生类，调用基类函数)
- 12.3.2 Aiming derived-class pointers at base-class objects (派生类指针指向基类，错误)
- 12.3.3 Derived-class member-function calls via base-class pointers (基类指针指向派生类，调用派生类函数，错误)
- 12.3.4 **virtual** functions (应用虚函数，解决上述问题)
- 12.3.5 Summary of the Allowed Assignments Between Base-Class and Derived-Class Objects and Pointers (基类/派生类对象和指针之间的赋值)

©2025, College of Software Engineering, Southeast University

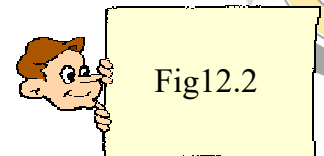
12.3.1 Invoking Base-Class Functions from Derived-Class Objects

- Base-class : `CommissionEmployee`
`void print() const;`
- Derived-class : `BasePlusCommissionEmployee`
`void print() const;`



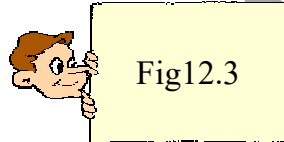
12.3.2 Aiming Derived-Class Pointers at Base-Class Objects

- Aim derived-class pointer at base-class object
 - Compilation error
 - `CommissionEmployee` (base-class object) is not a `BasePlusCommissionEmployee` (derived-class object)



12.3.3 Derived-Class Member-Function Calls via Base-Class Pointers

- Aiming base-class pointer at derived-class object
 - Should call base-class member
 - Invoked functionality depends on *handle* type
 - Calling functions that do not exist in base class (may exist only in derived class) will result in compilation error
 - However, this can be accomplished using **downcasting** (向下强制类型转换)



12.2.3 Derived-Class Member-Function Calls via Base-Class Pointers (cont.)

- Downcasting (§ 12.8)
 - If the address of a derived-class object has been assigned to a pointer of one of its direct or indirect base classes, it is acceptable to **cast** that base-class pointer **back** to a pointer of the derived-class type.

```
dynamic_cast < BasePlusCommissionEmployee* >
    (commissionEmployeePtr);
commissionEmployeePtr -> setBaseSalary(500);
```

- Question:
 - Which derived-class will base-class pointer be downcasted to ?

12.3.4 Virtual Functions

- Which class's function to invoke
 - Normally
 - *Handle* determines which class's functionality to invoke
 - With `virtual` functions
 - **Type of the object being pointed to**, not type of the handle, determines which version of a `virtual` function to invoke

12.3.4 Virtual Functions (cont.)

- Base-class:

```
class Animal {
public:
    virtual void move() const;
};
```

- Derived-class:

```
class Bird: public Animal {
    virtual void move() const;
};
```

Override (覆盖)

12.3.4 Virtual Functions (cont.)

- Virtual functions exist in base-classes and derived- classes in an inheritance hierarchy for polymorphism.
- virtual function in a derived-class must have the same **signature** and **return type** as the overridden virtual function in base-class.
- No keyword “virtual” needed in function definition.
- Example:

```
Bird b;  Animal*p = &b;
p->move();
```

©2025, College of Software Engineering, Southeast University

13

12.3.4 Virtual Functions (cont.)

- Invocation of virtual function
 - Dynamic binding (Late binding)
 - At execution time
 - Call by **pointer or reference of object** and **(->), (.)** operator
 - Polymorphism
 - Occurs only off **pointer** and **reference** handlers of base-class
 - Which member function to be invoked is decided according type of object going to be processed

©2025, College of Software Engineering, Southeast University

14

12.3.4 Virtual Functions (cont.)

- Invocation of virtual function
 - Static binding (Early binding)
 - At compile time
 - Called by **object name** and **(.)** operator

©2025, College of Software Engineering, Southeast University

15

12.3.4 Virtual Functions (cont.)

- Example:



Fig12.4~12.6

©2025, College of Software Engineering, Southeast University

16

12.3.4 Virtual Functions (cont.)

- Restrictions on virtual function
 - (1) Virtual functions must be member functions.
 - (2) Virtual functions cannot be static member functions.
 - (3) Constructor cannot be virtual.
 - (4) Destructor can be virtual.

©2025, College of Software Engineering, Southeast University

17

```
class Base {
public:
    Base(){ cout<< "Base"<< endl; p1 = new int(1); }
    virtual ~Base(){ cout<< "~Base"<< endl; delete p1; }
private:
    int *p1;
};
class Derive: public Base {
public:
    Derive(){ cout<<"Derive"<< endl; p2 = new int(2); }
    virtual ~Derive(){ cout<<"~Derive"<< endl; delete p2; }
private:
    int *p2;
};
int main() {
    Base* pDerive = new Derive;
    delete pDerive;
    return 0;
}
```

Base
Derive
~Derive
~Base

18

12.5 Abstract Classes and pure virtual Functions (cont.)

- Pure virtual function(纯虚函数)

- Specified by placing "= 0" in its declaration

```
virtual void move() const = 0;
```

- Do not provide implementation in base class

12.5 Abstract Classes and pure virtual Functions (cont.)

- Abstract classes (抽象类)

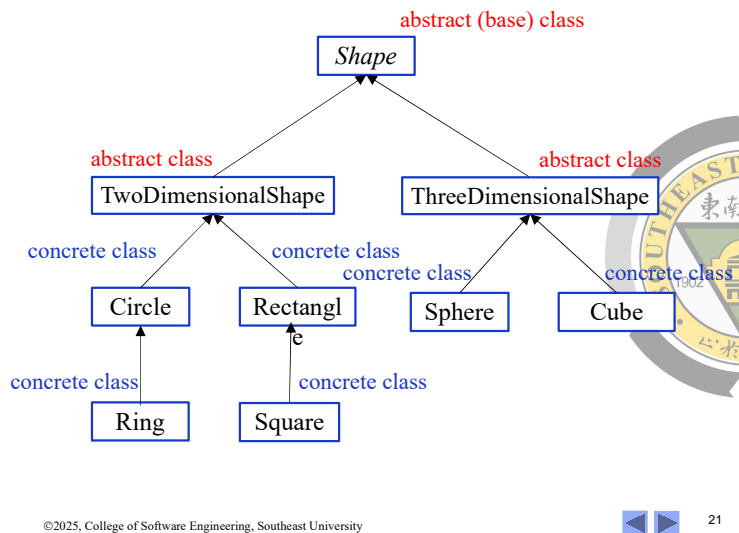
- Declares one or more pure virtual functions
- Cannot instantiate any object, but can declare pointer or reference
- Normally used as base classes

```
Animal obj;           // Error 不能实例化
Bird objBird;         // OK
Animal *ptr = &objBird; // OK 可指针
Animal &ref = objBird; // OK 可引用
```

作用：为派生类提供一个基本框架或公共接口。

- Concrete classes (具体类)

- Not including pure virtual function
- Can instantiate objects



12.5 Abstract Classes and pure virtual Functions (cont.)

- Conclusion:

- 成员函数是否声明为虚函数，取决于是否需要多态性支持。
- 虚函数是否声明为纯虚函数，取决于该函数对于当前类是否有意义，以及当前类是否需要实例化。

基类	派生类
虚函数 has an implementation	gives the derived class the option of overriding the function
纯虚函数 does not provide an implementation	requires the derived class to override the function (for that derived class to be concrete ; otherwise the derived class remains abstract)

12.6 Case Study: Payroll System Using Polymorphism

目的：输出各类员工的基本信息和薪水信息。

- Salaried employees(定薪雇员)

Name, SSN, Weekly Salary

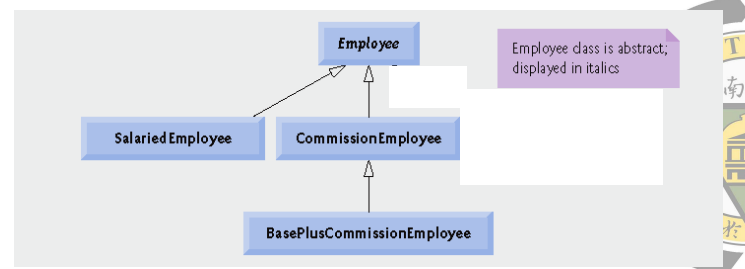
- Commission employees(佣金雇员)

Name, SSN, Gross sales amount, Commission rate

- Base-salary-plus-commission employees(带底薪佣金雇员)

Name, SSN, Gross sales amount, Commission rate, Base Salary

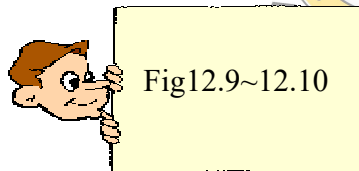
12.6 Case Study: Payroll System Using Polymorphism (cont.)



12.6.1 Creating Abstract Base Class Employee

• Employee Class

- Name, SSN: 各类员工的共有属性
- print(): 输出Name, SSN等基本信息 — 虚函数
- earnings(): 没有意义, 要求派生类实现 — 纯虚函数



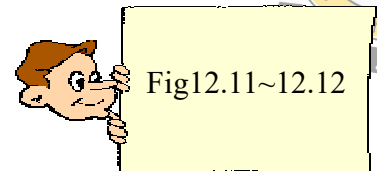
©2025, College of Software Engineering, Southeast University

25

12.6.2 Creating Concrete Derived Class SalariedEmployee

• Inherit from employee Class

- weeklySalary: 定薪雇员的独有属性
- print(): Override基类函数, 输出基本信息和薪水信息
- earnings(): 必须Override基类的纯虚函数, 计算薪水



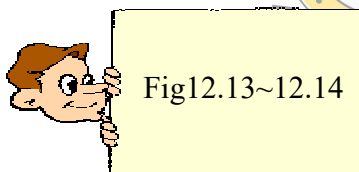
©2025, College of Software Engineering, Southeast University

26

12.6.3 Creating Concrete Derived Class CommissionEmployee

• Inherit from employee Class

- grossSales, commisionRate: 佣金雇员的独有属性
- print(): Override基类函数, 输出基本信息和薪水信息
- earnings(): 必须Override基类的纯虚函数, 计算薪水



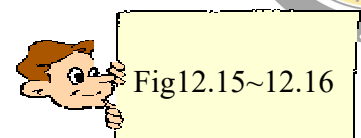
©2025, College of Software Engineering, Southeast University

27

12.6.4 Creating Indirect Concrete Derived Class BasePlusCommissionEmployee

• Directly inherit from CommissionEmployee Class, Indirectly inherit from Employee Class

- baseSalary: 带底薪佣金雇员的独有属性
- print(): Override基类(CommissionEmployee)函数, 输出基本信息和薪水信息
- earnings(): 选择Override基类的虚函数, 计算薪水



©2025, College of Software Engineering, Southeast University

28

12.6.6 Demonstrating Polymorphic Processing

• Create objects of four class types

- Manipulating objects with static binding
 - Using **name** handles
 - Compiler can identify each object's type to decide which **print** and **earnings** functions to call
- Manipulating objects polymorphically
 - Uses a vector of **Employee** pointers
 - Invoke **virtual** functions using **pointers** and **references**
- See in *Fig12.17*

©2025, College of Software Engineering, Southeast University

29